

MOBILE APPLICATION SECURITY TESTING

Test Target: termux-app_v0.118.0+github-
debug_x86_64.apk

TERMUX ANDROID APPLICATION

SHIJIMOL S
NOVAVI STUDENT

18-03-2026

Contents

1. Executive Summary	3
1.2 Key Findings	3
1.2.1 Severity	4
1.2.2 Vulnerability Severity Distribution	4
1.2.3 Issue Categorization	5
2. Vulnerability Details	6
2.1 High Severity Findings	6
2.1.1 Root Detection Not Implement	6
2.2 Medium Severity Findings	8
2.2.1 Application Running in Debug Mode	8
2.2.2 Exported Content Provider Protected by Weak Permission	9
2.2.3 Exported Command Execution Service Protected by Weak Permission	11
2.2.4 Weak Protection Level for Command Execution Permission	12
2.3 Low Severity Findings	13
2.3.1 Use of SYSTEM_ALERT_WINDOW (Overlay Permission)	13
2.3.2 Application Requests Permission to Install Packages	15
2.3.3 Exported Settings Activity	16
2.4 Informational Findings	17
2.4.1 Non-Exported Activity Reducing Attack Surface	17
2.4.2 Standard Activity Configuration Parameters	19
3. Conclusion	20

1.1 Executive Summary

In March 2026, a Mobile Application VAPT was conducted on the Android application Termux (Debug Build) to evaluate its security posture and identify potential vulnerabilities affecting confidentiality, integrity, and availability.

The assessment included static analysis, reverse engineering, permission analysis, and dynamic testing. A total of 10 findings were identified:

- High: 1
- Medium: 4
- Low: 3
- Informational: 2

The key issues include weak permission protection, exported components, and debug mode configuration, which may lead to unauthorized access and increased attack surface.

No Critical vulnerabilities were identified. Overall, the risks are mainly due to misconfigurations rather than complex exploits, indicating a moderate security risk.

1.2 Key Findings

The security assessment of the Termux Android application identified multiple vulnerabilities across different severity levels. The majority of issues are related to improper component exposure, weak permission configurations, and debug build settings.

The most critical observations include:

- The application is distributed as a debug build, which exposes internal debugging capabilities and increases the risk of reverse engineering.
- A custom permission (`RUN_COMMAND`) is configured with a dangerous protection level, allowing third-party applications to request access to sensitive functionality.
- Exported components (Service and Content Provider) are protected by weak permissions, potentially enabling unauthorized interaction and command execution.
- The application requests high-risk permissions such as `SYSTEM_ALERT_WINDOW` and `REQUEST_INSTALL_PACKAGES`, increasing the attack surface.
- Some components are securely configured (e.g., non-exported activities), indicating partial adherence to secure development practices.

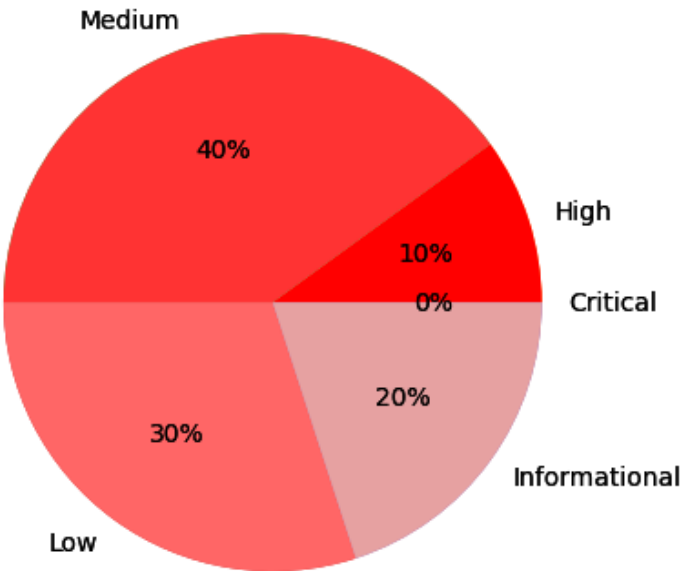
Overall, the findings indicate that the application has moderate security risks, primarily due to misconfigurations rather than complex vulnerabilities.

1.2.1 Severity

Severity	# of Findings
Critical	0
High	1
Medium	4
Low	3
Informational	2

Table 1: Severity of Vulnerabilities

1.2.2 Vulnerability Severity Distribution



1.2.3 Issue Categorization

Issue Category	# of Findings
Improper Permission Configuration	2
Exported Components / Component Exposure	3
Insecure Build Configuration (Debug Mode)	1
Risky / Excessive Permissions	2
Secure Configuration / Best Practices	2

Chapter 1

Vulnerability Details

1.1 High Severity

1.1.1 Root Detection Not Implement

Severity: **HIGH**

Description

During dynamic analysis, the Termux application was installed and executed within a controlled Android emulator environment (Genymotion). The application successfully launched and provided a terminal interface that allows interaction with the Android filesystem.

The tester navigated through directories using standard Linux commands such as `pwd`, `cd`, and `ls` to observe the accessible file system structure.

This step helps evaluate whether the application allows unauthorized access to system directories or sensitive resources.

Discovery Method

Dynamic Analysis using tools like Android Debug Bridge (ADB), Genymotion Android Emulator, and manual command execution within the Termux terminal.

Impact Analysis

- Unauthorized command execution within the application environment
- Access to accessible filesystem directories
- Information disclosure if sensitive files are readable
- Potential privilege misuse if combined with other vulnerabilities

Proof of Concept

1. Install the target APK (`termux-app_v0.118.0+github-debug_x86_64.apk`) on the Android emulator (Genymotion).
2. Launch the Termux application from the device launcher.
3. Once the terminal interface appears, execute the following commands.

`pwd`

Output:

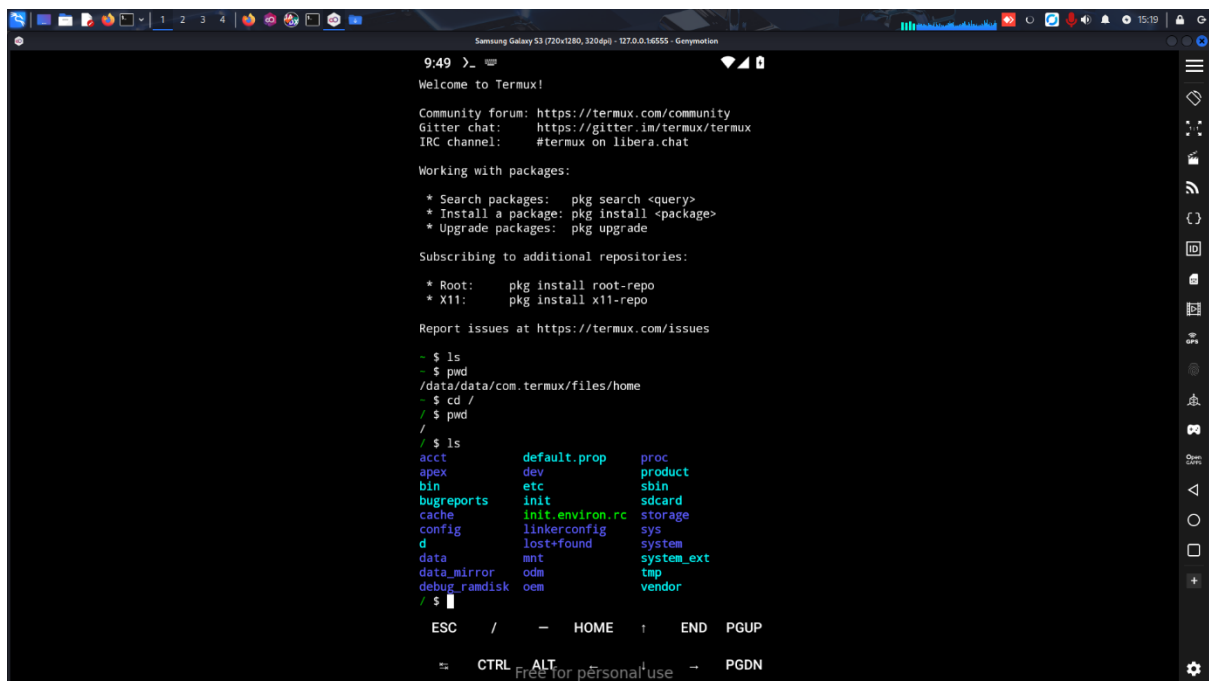
```
/data/data/com.termux/files/home
```

4. Navigate to the root directory.

```
cd /
```

5. List the available directories.

```
ls
```



```
Samsung Galaxy S3 (720x1280, 320dpi) - 127.0.0.16555 - Genymotion
9:49
Welcome to Termux!

Community forum: https://termux.com/community
Gitter chat: https://gitter.im/termux/termux
IRC channel: #termux on libera.chat

Working with packages:

* Search packages: pkg search <query>
* Install a package: pkg install <package>
* Upgrade packages: pkg upgrade

Subscribing to additional repositories:

* Root: pkg install root-repo
* X11: pkg install x11-repo

Report issues at https://termux.com/issues

~ $ ls
~ $ pwd
/data/data/com.termux/files/home
~ $ cd /
/ $ pwd
/
/ $ ls
acct      default.prop  proc
apex      dev           product
bin       etc           sbin
bugreports  init         sdcard
cache     init.environ.rc storage
config    linkerconfig  sys
d         lost+found    system
data      mnt           system_ext
data_mirror  odm          tmp
debug_ramdisk oem          vendor

ESC / - HOME ↑ END PGUP
CTRL ALT Free for personal use PGDN
```

Figure 1 – Termux Application Runtime Showing Filesystem Access in Genymotion Emulator

Mitigation

- Restrict Sensitive Permissions
- Enforce Android Sandbox Restrictions
- Input Validation for Command Execution
- Limit Access to System Resources

1.2 Medium Findings

1.2.1 Application Running in Debug Mode

Severity: **MEDIUM**

Description

The Android application is compiled with the `android:debuggable` flag enabled in the `AndroidManifest.xml` file. When an application is debuggable, attackers can attach debugging tools to analyze application behavior during runtime.

Debug mode allows access to internal application processes, memory, and debugging information, which may aid attackers in reverse engineering the application and bypassing security mechanisms.

This configuration should never be enabled in production applications.

Discovery Method

Static Analysis

The vulnerability was identified during the analysis of the `AndroidManifest.xml` file using reverse engineering tools such as:

- APKTool
- JADX

Proof of Concept (PoC)

1. Decompile the APK file using APKTool:

```
apktool d termux-app.apk
```

2. Open the extracted folder.
3. Locate and open the file:

```
AndroidManifest.xml
```

4. Observe the following configuration:

```
android:debuggable="true"
```

This indicates that the application is built with debugging enabled.

Impact Analysis

- Attach a debugger to the application
- Inspect application memory
- Analyze runtime behavior
- Bypass certain security controls
- Extract sensitive data

Mitigation

- Disable debugging in production builds by setting the following configuration:

```
android:debuggable="false"
```

- Ensure release builds disable debugging automatically.
- Use ProGuard or code obfuscation to make reverse engineering more difficult.

1.2.2 Exported Content Provider Protected by Weak Permission

Severity: **MEDIUM**

Description

The Android application exposes a content provider

`com.termux.app.TermuxOpenReceiver.ContentProvider` with the configuration `android:exported="true"` in the `AndroidManifest.xml`.

Exported content providers allow external applications to interact with the application via content URIs. In this case, access to the provider is protected by a custom permission `com.termux.permission.RUN_COMMAND`.

However, the permission is defined with the protection level **dangerous**, which allows third-party applications to request and obtain this permission from the user at runtime.

Due to this weak protection mechanism, a malicious application could potentially gain access to the content provider and interact with exposed data or functionality.

Discovery Method

Static Analysis

The issue was identified during manual review of the `AndroidManifest.xml` file after decompiling the APK using:

- APKTool
- JADX

Proof of Concept (PoC)

1. The application was installed on an Android emulator.
2. The exported content provider was identified in the `AndroidManifest.xml`:

```
android:authorities="com.termux.files"
```

3. The provider was accessed using ADB:

```
adb shell content query --uri content://com.termux.files
```

4. The command attempts to interact with the exported content provider.

❖ Evidence found in `AndroidManifest.xml`:

```
<provider
    android:name="com.termux.app.TermuxOpenReceiver.ContentProvider"
    android:permission="com.termux.permission.RUN_COMMAND"
    android:exported="true"
    android:authorities="com.termux.files"
    android:grantUriPermissions="true"/>
```

Impact Analysis

If exploited, a malicious application could:

- Access or query data exposed by the content provider
- Interact with file handling functionality
- Abuse application components through crafted content URIs

The impact depends on the operations supported by the provider.

Mitigation

To reduce the risk, developers should:

1. Restrict the content provider from external access if not required:

```
android:exported="false"
```

2. Strengthen the permission protection level:

```
android:protectionLevel="signature"
```

3. Validate all incoming content URI requests before processing.
4. Avoid exposing sensitive data through content providers.

1.2.3 Exported Command Execution Service Protected by Weak Permission

Severity: **MEDIUM**

Description

The Android application exposes a service `com.termux.app.RunCommandService` with the configuration `android:exported="true"` in the `AndroidManifest.xml`.

This service appears to provide functionality for executing commands in the Termux environment. Although access to the service is restricted by the custom permission `com.termux.permission.RUN_COMMAND`, the permission is defined with the protection level **dangerous**, allowing third-party applications to request it.

As a result, a malicious application installed on the device could request this permission and potentially interact with the exported service to execute commands through the Termux application.

Discovery Method

Static Analysis

The issue was discovered during manual review of the `AndroidManifest.xml` file after decompiling the APK using:

- APKTool
- JADX

Proof of Concept (PoC)

An attacker application with the `RUN_COMMAND` permission could attempt to trigger the service using the Android Activity Manager.

Example command:

```
adb shell am startservice -n com.termux/.app.RunCommandService
```

If the permission is granted, the service may process the request and execute commands.

Impact Analysis

- Execute commands within the Termux environment
- Abuse application functionality
- Run scripts or commands without user awareness

Mitigation

- Protect the permission with signature-level protection.
- Avoid exporting services that execute commands unless absolutely necessary.
- Implement strict validation of all incoming intents before executing commands.

1.2.4 Weak Protection Level for Command Execution Permission

Severity: **MEDIUM**

Description

The Android application defines a custom permission `com.termux.permission.RUN_COMMAND` with the protection level set to dangerous. Permissions with a dangerous protection level can be requested by any third-party application and granted by the user at runtime.

This permission is used to control access to the `RunCommandService`, which allows command execution within the Termux environment. If a malicious application obtains this permission, it may interact with the exported service and trigger command execution inside the Termux application.

Improper protection of such sensitive functionality increases the risk of unauthorized command execution and abuse of application capabilities.

Discovery Method

Static Analysis

The vulnerability was identified during analysis of the `AndroidManifest.xml` file using reverse engineering tools such as:

- APKTool
- JADX

The manifest file revealed that the custom permission used to protect command execution functionality is configured with a dangerous protection level.

Proof of Concept PoC

1. Open the AndroidManifest.xml file.
2. Locate the custom permission definition:

Evidence found in AndroidManifest.xml:

```
<permission  
  
    android:name="com.termux.permission.RUN_COMMAND"  
  
    android:protectionLevel="dangerous"/>
```

3. Observe that the permission protecting the command execution service can potentially be requested by external applications.

Impact Analysis

- Unauthorized command execution
- Abuse of application functionality
- Execution of malicious scripts
- Increased attack surface for the application

Mitigation

- The permission protection level should be restricted to signature
- Restrict exported services that allow command execution
- Validate all incoming intents
- Follow the principle of least privilege

1.3 Low Findings

1.3.1 Use of SYSTEM_ALERT_WINDOW (Overlay Permission)

Severity: **LOW**

Description

The Android application requests the permission `android.permission.SYSTEM_ALERT_WINDOW` in the AndroidManifest.xml.

This permission allows the application to draw overlay windows on top of other applications. While this capability may be required for certain functionalities, it is considered a high-risk permission because it can be abused to display deceptive interfaces over legitimate applications.

If misused, overlay windows can be used to perform clickjacking attacks, phishing attacks, or UI spoofing, potentially tricking users into interacting with malicious content.

Discovery Method

Static Analysis

The permission was identified during manual review of the `AndroidManifest.xml` file after decompiling the APK using:

- APKTool
- JADX

Proof of Concept (PoC)

- Install and launch the application on an Android device or emulator.
- Navigate to the system settings and observe that the application requests permission to display over other apps.
- Verify Permission Declaration (Static Check)
Decompile the APK and confirm the permission in `AndroidManifest.xml`:

```
apktool d termux-app.apk
```

Locate:

```
<uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>
```

- If granted, the application can create overlay windows that appear above other applications.

Impact Analysis

If improperly implemented or abused, this permission could allow:

- Display of deceptive overlays over legitimate applications
- Clickjacking attacks to capture user interactions
- Phishing attacks by mimicking trusted interfaces
- Interference with normal user interaction

However, no direct exploitation was observed during testing.

Mitigation

- Avoid requesting `SYSTEM_ALERT_WINDOW` unless absolutely necessary
- Clearly inform users why the permission is required
- Restrict overlay usage to trusted and controlled scenarios
- Implement safeguards to prevent misuse of overlay functionality

1.3.2 Application Requests Permission to Install Packages

Severity: LOW

Description

The Android application requests the permission

`android.permission.REQUEST_INSTALL_PACKAGES` in the `AndroidManifest.xml`.

This permission allows the application to prompt the installation of APK files from unknown sources. While this functionality may be required for certain use cases, it can introduce security risks if misused.

A malicious or compromised application could potentially leverage this permission to install unauthorized or malicious applications on the user's device, leading to further compromise.

Discovery Method

Static Analysis

The permission was identified during manual review of the `AndroidManifest.xml` file after decompiling the APK using:

- APKTool
- JADX

Proof of Concept (PoC)

1. Install the Application

Install the APK on an Android emulator or device.

2. Verify Permission in Manifest

Decompile the APK:

```
apktool d termux-app.apk
```

Locate in `AndroidManifest.xml`:

```
<uses-permission  
android:name="android.permission.REQUEST_INSTALL_PACKAGES"/>
```

3. Trigger APK Installation Flow (Conceptual)

If the application includes functionality to install APKs:

- Attempt to download or open an APK file within the application
- Observe whether the application prompts installation

4. Check System Behavior

- Android prompts the user to allow installation from unknown sources
- The application can initiate APK installation requests

Impact Analysis

If abused, this permission could allow:

- Installation of malicious applications
- Unauthorized software deployment
- Increased risk of device compromise

Mitigation

- Avoid requesting `REQUEST_INSTALL_PACKAGES` unless absolutely necessary
- Restrict APK installation features to trusted sources only
- Clearly inform users before initiating installation requests
- Validate all downloaded or external APK files before installation

1.3.3 Exported Settings Activity

Severity: **LOW**

Description

The Android application exposes the activity `com.termux.app.activities.SettingsActivity` with the configuration `android:exported="true"` in the `AndroidManifest.xml` file.

Exported activities can be launched by external applications through Android intents. If proper access control or input validation is not implemented within the activity, malicious applications could potentially invoke the activity and interact with application settings.

Although settings activities are sometimes intentionally exported, exposing them without permission protection increases the attack surface of the application.

Discovery Method

Static Analysis

The configuration was identified during manual review of the AndroidManifest.xml file after decompiling the APK using:

- APKTool
- JADX

Configuration found in AndroidManifest.xml:

```
<activity
    android:name="com.termux.app.activities.SettingsActivity"
    android:exported="true"/>
```

Proof of Concept (PoC)

1. Install the target application on an Android emulator or test device.
2. Connect to the device using **ADB (Android Debug Bridge)**.

```
adb devices
```

3. Attempt to start the exported activity using the following command:

```
adb shell am start -n com.termux/.app.activities.SettingsActivity
```

4. Observe that the activity launches successfully.

Mitigation

If external access is not required, restrict the activity by disabling export:

```
android:exported="false"
```

Alternatively, protect the activity using a **custom permission** to ensure that only trusted applications can launch it.

1.4 Informational Findings

1.4.1 Non-Exported Activity Reducing Attack Surface

Severity: INFORMATIONAL

Description

The Android application defines certain activities with the configuration `android:exported="false"` in the `AndroidManifest.xml`.

This setting ensures that the activity cannot be accessed or launched by external applications through Android intents. Restricting component exposure is a recommended security practice, as it limits unauthorized interaction with internal application components.

Discovery Method

Static Analysis

The configuration was identified during manual review of the `AndroidManifest.xml` file after decompiling the APK using:

- APKTool
- JADX

Proof of Concept (PoC)

1. Attempt to launch the activity externally using ADB:

```
adb shell am start -n com.termux/.app.activities.HelpActivity
```

2. Observe that the activity cannot be launched externally due to `exported="false"`.

❖ Evidence

Example activity configuration:

```
<activity
    android:name="com.termux.app.activities.HelpActivity"
    android:exported="false"/>
```

Impact Analysis

This configuration prevents external applications from invoking internal activities, thereby reducing the risk of:

- Unauthorized activity access
- Intent-based attacks
- Exposure of internal functionality

No security risk was identified.

1.4.2 Standard Activity Configuration Parameters (UI and Lifecycle Settings)

Severity: INFORMATIONAL

Description

The Android application defines activities with standard configuration parameters such as `launchMode="singleTask"` and `configChanges` within the `AndroidManifest.xml`.

These configurations are used to control the activity lifecycle and behavior during runtime events such as screen rotation, keyboard visibility changes, and multi-window support. They are part of normal Android application design and do not introduce any security vulnerabilities.

Discovery Method

Static Analysis

The configuration was identified during manual review of the `AndroidManifest.xml` file after decompiling the APK using:

- APKTool
- JADX

❖ Evidence

Example activity configuration:

```
<activity
  android:name="com.termux.app.TermuxActivity"
  android:launchMode="singleTask"
  android:configChanges="density|screenSize|orientation|keyboardHidden"/>
```

Chapter 2

Conclusion

The Mobile Application Security Assessment conducted on the Termux Android application revealed several security weaknesses primarily related to misconfigurations and insecure implementation practices.

A total of 10 findings were identified, including:

- 1 High severity
- 4 Medium severity
- 3 Low severity
- 2 Informational

No Critical vulnerabilities were identified during the assessment.

The most significant risks stem from:

- Use of debuggable build configuration, which exposes internal application behavior
- Weak custom permission protection, allowing potential misuse of command execution functionality
- Exposure of application components (services and content providers) that can be accessed externally

Additionally, the presence of high-risk permissions such as `SYSTEM_ALERT_WINDOW` and `REQUEST_INSTALL_PACKAGES` increases the overall attack surface of the application.

However, it is important to note that the application is a terminal emulator, and certain behaviors (such as command execution and filesystem access) are part of its intended functionality and do not represent security vulnerabilities.

The identified issues do not indicate advanced exploitation techniques but rather highlight lack of secure configuration and adherence to best practices.

Final Security Assessment

- The application is moderately vulnerable
- Risks are mainly due to configuration weaknesses
- No evidence of critical or advanced exploitation paths

Recommendations Summary

To improve the security posture of the application, the following actions are recommended:

- Disable debug mode in production builds
- Change custom permissions to signature protection level

- Restrict or properly secure exported components
- Apply the principle of least privilege for permissions
- Conduct regular security testing before release

The application demonstrates functional correctness but lacks secure deployment practices. Addressing the identified issues will significantly reduce the attack surface and improve the application's overall security posture.